Chord DHT Project Tutorial

1. Background: What is Chord?

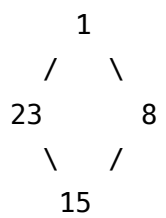
Chord is a classic **Distributed Hash Table (DHT)** protocol. Imagine a group of computers forming a "ring," where each computer (node) is responsible for storing a portion of the data. The core question Chord answers is: **Given a key, how do we quickly find which node stores it?**

Key Concepts

- **Ring Identifier Space**: All nodes and keys are mapped onto a ring of size `[0, 2^m - 1]` (in this project `m=5`, so the ring size is 32).
- **Successor**: The closest node clockwise from a key on the ring — this node is responsible for storing that key.
- **Predecessor**: The closest node counter-clockwise from the current node.
- **Finger Table**: Each node maintains a routing table of size `m`. Entry `i` points to the successor of `(node_id + 2^(i-1)) mod 2^m`. This reduces lookup complexity from O(N) to **O(log N)**.

Intuitive Example

Suppose the ring has nodes `1, 8, 15, 23` (m=5, ring size 32):



The successor of key `10` is node `15` (the first node ≥ 10 going clockwise).

2. Project Structure and Workflow

You only need to edit `chord_node.py`, implementing the methods marked with `TODO` in the `Node` class. **Do not modify** `test_dht.py`.

Overall Execution Flow

```
create() → join() → [stabilize() + fix_fingers()] × many rounds → put()/get() → transf
```

1. The first node calls `create()` to form the ring
2. Subsequent nodes call `join()` to enter the ring
3. Repeated calls to `stabilize()` and `fix_fingers()` converge the ring to a correct state
4. Use `put()` / `get()` to store and retrieve data
5. When a new node joins, `transfer_keys()` redistributes data

3. Step-by-Step Implementation Guide

Part 1: Core Protocol (4 points)

3.1 `in_interval(id, start, end, inclusive_right=True)`

A utility method used by nearly every other method.

Questions to consider:

- On a regular number line, checking `id ∈ (start, end]` is straightforward. But on a ring?
- When `start > end` (e.g., `start=28, end=3`), the interval wraps around 0. How do you handle this?
- What does `start == end` mean? (Hint: the entire ring)

3.2 `find_successor(id)`

Find the successor of `id` — the most fundamental Chord operation.

Algorithm outline:

1. If `id` falls in `(self.node_id, self.successor]`, return the successor directly
2. Otherwise, find the closest preceding node from the finger table, and ask **that node** to continue the search

Key point: Step 2 requires `remote_call` — because that node is not yourself.

3.3 `closest_preceding_node(id)`

Scan the finger table from **back to front** (index `m` down to `1`). Return the first finger whose `node_id` falls in the **open interval** `(self.node_id, id)`.

Think about: Why scan from back to front? (Hint: `finger[m]` covers the largest range)

3.4 `create()` and `join(existing_node_addr)`

- `create()` : Very simple — this is the first node in the ring. Successor points to self, predecessor is None.
- `join()` : Set predecessor to None, and ask the existing node to find your successor.

Hint: In `join()` , you need to use `remote_call` to send a `"find_successor"` request to the existing node.

3.5 `stabilize()`

The stabilization protocol:

1. Ask your successor: "Who is your predecessor?" (use `remote_call` + `"get_predecessor"`)
2. If that predecessor is between you and your successor, it should be your new successor — update accordingly
3. Notify your (possibly new) successor: "I might be your predecessor" (use `remote_call` + `"notify"`)

3.6 `notify(node_addr)`

When another node tells you "I might be your predecessor":

- If you have no predecessor, accept it
- If it truly falls in `(self.predecessor, self)` , accept it

3.7 `fix_fingers()`

Each call fixes one finger table entry:

- `finger[i]` should be the successor of `(node_id + 2^(i-1)) mod 2^m`
- Use `self.next` to track which index to fix, incrementing (and wrapping back to 1) after each call

Part 2: Key Storage (4 + 2 points)

3.8 `hash_key(key)`

Map a string key to an integer ID on the ring.

Hint: Use `hashlib.sha1()` and take `mod 2^m` .

3.9 `put(key, value)` and `get(key)`

Common pattern:

1. Use `hash_key()` to get the key's ID
2. Use `find_successor()` to find the responsible node
3. Use `remote_call` to send a `"store"` or `"retrieve"` command to that node

3.10 `transfer_keys(new_node_addr)`

After a new node joins, its successor must hand over keys that now belong to the new node.

Questions to consider:

- Which keys should be transferred? (Hint: those whose `hash_key(key)` falls within the new node's range)
- What range is the new node responsible for? (Hint: `(new_node's predecessor, new_node_id]` — think about what the predecessor is in this context)
- Don't forget to remove transferred keys from `self.data`

Part 3: Failure Recovery (Bonus, 2 points)

3.11 `check_predecessor()`

Use `remote_call` to send a `"ping"` to your predecessor. If an exception is raised, the predecessor has failed — set it to `None`.

Hint: Use `try/except` to catch the exception.

4. Debugging Tips

1. **Start with the simplest test**: Make sure `create_chord_network` passes first.
2. **Print debugging**: Add `print` statements in key methods to trace node IDs and lookup paths.
3. **Understand the test flow**: Carefully read what each test function in `test_dht.py` does.
4. **Draw it out**: Sketch the ring and node positions on paper. Manually simulate `find_successor`.
5. **Check `remote_call` parameter formats**: Look at how `handle_client` parses parameters for each command.

5. Running Tests

```
python test_dht.py
```

Save output:

```
python test_dht.py > output_dht.txt 2>&1
```